

OE-EC704A — Web Technology

Semester VII • 3L:0T:0P • 3 Credits

Module 5 Exam Notes: Applets, Event Handling & AWT Controls

CGEC | B.Tech ECE (2023–27) | *Biraj Sarkar*

Contents

1	Java Applets	2
1.1	Applet Life Cycle	2
1.2	Applet Display and Status Methods	2
1.3	HTML Applet Tag and Parameters	3
1.4	Applet vs. Standard Application	4
2	Delegation Event Model	4
2.1	Adapter Classes	4
3	AWT (Abstract Window Toolkit)	5
3.1	AWT vs. Swing	5
3.2	Layout Managers	5

1. Java Applets

An applet is a specialized Java program designed to be embedded within an HTML web page and executed on a client machine inside a web browser sandbox.

Applet Security Sandbox

Applets run in a restricted runtime environment (sandbox) to prevent malicious behaviors:

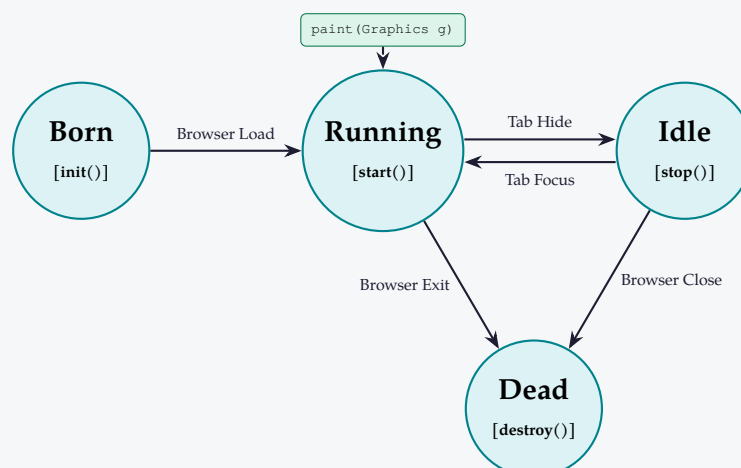
- Applets cannot read or write files on the client local disk.
- Applets cannot open network connections to any host except the one they were downloaded from.
- Applets cannot start external execution processes or execute native system code.

1.1 Applet Life Cycle

An applet defines five lifecycle callback methods managed by the browser JVM:

- **init ()**: Invoked once during loading to initialize variables and load GUI elements.
- **start ()**: Invoked after initialization or when visiting the page to resume execution.
- **paint (Graphics g)**: Invoked when the applet display needs redrawing.
- **stop ()**: Invoked when navigating away from the page to pause background threads.
- **destroy ()**: Invoked before unloading the applet to release system resources.

Applet Life Cycle State Transitions



1.2 Applet Display and Status Methods

Applets provide methods to interact with their display canvas and output messages directly to the hosting browser's status window:

- **Foreground and Background Control**: Inherited from the `Component` class.

- `void setBackground(Color c)`: Sets the background color of the applet window.
- `void setForeground(Color c)`: Sets the default color for text and graphical drawings inside the applet.
- `Color getBackground()` and `Color getForeground()`: Retrieve the current background and foreground colors.
- **Status Window Output:** The browser running the applet contains a status bar (status window).
 - `void showStatus(String message)`: Displays the specified message string in the browser's status window. This is useful for user feedback (e.g., displaying coordinates, loading percentages, or event responses).

1.3 HTML Applet Tag and Parameters

Applets are executed by inserting them into an HTML document using the `<applet>` tag. The browser reads the tag, downloads the compiled bytecode, and runs it inside the local JVM.

HTML `<applet>` Tag Syntax and Attributes

```
<applet codebase="classes/" code="MyApplet.class" width="400" height="300">  
  <param name="userName" value="Biraj">  
  <param name="sampleRate" value="50">  
  Alt text: Your browser does not support Java Applets.  
</applet>
```

Core attributes of the `<applet>` tag:

- **code**: Specifies the filename of the compiled applet class (mandatory).
- **width & height**: Specify the initial display area dimensions in pixels (mandatory).
- **codebase**: Specifies the base URL or directory path containing the class files (optional).
- **alt**: Specifies fallback text to display if the browser supports the tag but cannot run it.

Passing Parameters via the `<param>` Tag:

- Parameters are defined in HTML using the `<param name="..." value="...">` tag inside the applet block.
- Inside the applet's `init()` method, these parameters are retrieved as Strings using: `String val = getParameter("userName");`

1.4 Applet vs. Standard Application

Feature	Applet Class	Standard Java Application
Execution Target	Embedded in HTML; runs inside browser JVM.	Standalone execution on the client OS.
Main Entry Method	No <code>main()</code> entry. Uses lifecycle methods.	Requires a <code>public static void main</code> entry.
Security Rules	Heavily sandboxed (restricted access).	Full access to local hardware and file systems.
Superclass Class	Must extend <code>java.applet.Applet</code> .	Does not require a specific superclass.

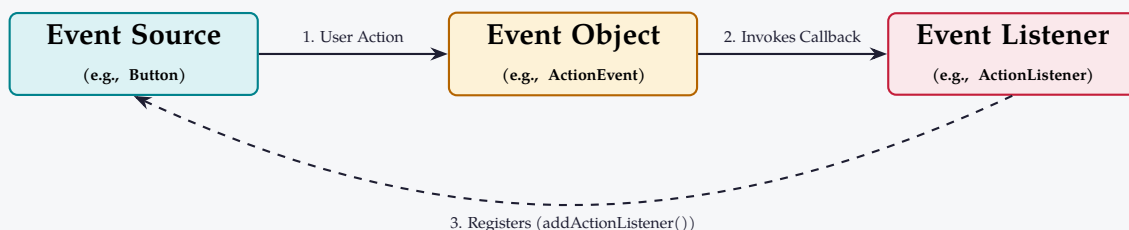
2. Delegation Event Model

Java event handling is built on the **Delegation Event Model**, which separates the user interface controls from the logic handling user events.

Model Components

- **Event Source:** The GUI component that generates the event when acted upon (e.g., `Button`).
- **Event Object:** Encapsulates details of the event state change (e.g., `ActionEvent`).
- **Event Listener:** An interface target that receives and processes event notifications.

Delegation Event Model Architecture



2.1 Adapter Classes

An event listener interface may define multiple abstract methods (e.g., `WindowListener` defines seven). Implementing it directly forces you to provide empty overrides for all of them.

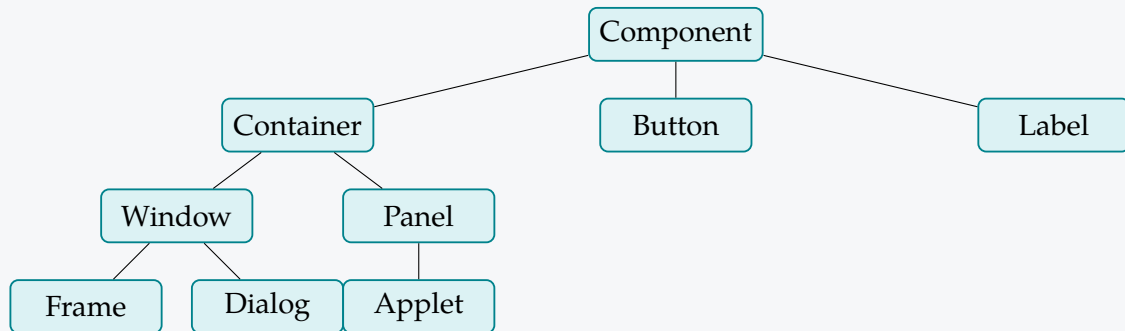
Adapter Class Solution

An **Adapter Class** is a helper class that implements a listener interface with empty default method overrides. By extending the adapter class (e.g., extending `WindowAdapter`), you only override the specific callback methods you need.

3. AWT (Abstract Window Toolkit)

AWT is Java's original platform-dependent GUI library. It uses native system peers to render controls, resulting in a look-and-feel that matches the host OS.

AWT Component Class Hierarchy



3.1 AWT vs. Swing

Metric	AWT (Heavyweight)	Swing (Lightweight)
Rendering Engine	Uses host OS peer components.	Rendered purely in Java (custom painting).
Platform Style	Platform-dependent look-and-feel.	Platform-independent look-and-feel.
Speed	Fast (delegates rendering to host).	Slower (rendered by JVM engine).
Class Package	Lives in <code>java.awt.*</code>	Lives in <code>javax.swing.*</code>

3.2 Layout Managers

Layout managers control the sizing and positioning of child components inside a container:

- **FlowLayout (Default for Panel/Applet):** Places components in a row, wrapping when the boundary is reached.
- **BorderLayout (Default for Frame/Window):** Places components in five regions: North, South, East, West, Center.
- **GridLayout:** Places components in a uniform grid of equal-sized rows and columns.

Quick Revision — Module 5

Applets, Event Handling & AWT Controls | OE-EC704A

HTML Applet & Event Handling Syntax

Concept	Syntax and Example Usage
HTML Applet Tag	<code><applet code="MyApplet.class" width="300" height="200"></applet></code>
Applet Parameters	<code><param name="message" value="Hello World"></code>
Parameter Query	<code>String val = getParameter("message");</code>
Applet paint Override	<code>public void paint(Graphics g) { g.drawString("Test", 20, 20); }</code>
AWT & Events	Registration & Design Declarations
AWT Button Registration	<code>Button btn = new Button("Submit"); btn.addActionListener(listener);</code>
Anonymous Inner Class	<code>btn.addActionListener(new ActionListener() { ... });</code>
Set Frame Layout	<code>setLayout(new BorderLayout()); add(btn, BorderLayout.NORTH);</code>
AWT Frame Show	<code>Frame f = new Frame("Title"); f.setSize(300, 300); f.setVisible(true);</code>

One-Line Definitions

- **Applet:** A small Java program executed inside a web browser container.
- **Delegation Event Model:** An event framework separating event detection from handler logic.
- **Event Source:** A UI component that registers listeners and fires events on state changes.
- **Event Listener:** An interface declaring callback methods to handle specific events.
- **Adapter Class:** A helper class with default empty implementations of a listener interface.
- **Anonymous Inner Class:** An unnamed inline class used to implement listener interfaces.
- **AWT:** A platform-dependent GUI library utilizing host OS peers for rendering.
- **Layout Manager:** A layout interface deciding the spatial distribution of child components.

Frequently Asked / Viva Questions**Q1. Which class does every Java Applet extend? Which package contains it?**

Every Applet extends `java.applet.Applet` (or `javax.swing.JApplet` for Swing). The class resides inside the `java.applet` package.

Q2. Why does an applet not contain a `main()` method?

Applets do not run as standalone applications. They are executed by a web browser container, which handles loading and calls their lifecycle methods.

Q3. What is the difference between `init()` and `start()` methods?

`init()` is called once when the applet is first loaded to handle initialization. `start()` is called every time the user visits or returns to the page.

Q4. What is the purpose of the `paint(Graphics g)` method? Who calls it?

The `paint()` method renders the applet graphics (text, lines, shapes) on the screen. It is called by the browser JVM when the applet first displays or needs redrawing.

Q5. How do you pass parameters from HTML to an Applet?

Using the `<param>` tag inside the `<applet>` tag in HTML, and querying it in the applet using the `getParameter(String name)` method.

Q6. What is the difference between `getCodeBase()` and `getDocumentBase()`?

`getCodeBase()` returns the URL of the directory containing the applet's compiled class files. `getDocumentBase()` returns the URL of the HTML document embedding the applet.

Q7. Why are AWT components referred to as heavyweight controls?

AWT controls are coupled to the underlying operating system. Each AWT component is associated with a native peer window, consuming significant memory and OS resources.

Q8. What is the purpose of an Adapter class in event handling?

It simplifies listener implementation. By extending an adapter class, you only override the specific event handler methods you need, rather than providing empty overrides for all methods.

Q9. How does `FlowLayout` arrange components? What is its default alignment?

`FlowLayout` arranges components in a single line from left to right. When it runs out of space, it wraps them to the next line. Its default alignment is centered.

Q10. Why did AWT become deprecated in favor of Swing?

AWT's platform-dependent rendering led to inconsistent layouts across operating systems. Swing introduced lightweight, platform-independent controls rendered entirely in Java.

Applet Lifecycle Methods Transition Matrix

Method	Invocation Frequency	Primary Purpose
init ()	Invoked once when first loaded.	Initializes variables, loads fonts, images, and GUI elements.
start ()	Invoked on load and page returns.	Resumes applet execution, starts animations or threads.
paint ()	Invoked on display and redraw requests.	Renders text, shapes, and images to the screen coordinate system.
stop ()	Invoked on page exit or tab switch.	Suspends threads, pauses animations to save CPU resources.
destroy ()	Invoked once before unloading.	Reclaims resources, kills threads, and cleans up memory.